

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

October 20, 2016 · GAN KL DIVERGENCE
GENERATIVE MODELS

Instance Noise: A trick for stabilising GAN training

with [Casper Kaae Sønderby]
(<https://casperkaae.github.io/>)

Generative Adversarial Networks (GANs) are notoriously hard to train. In a recent paper, we presented an idea that might help remedy this.

Our intern **Casper** spent the summer working with GANs, resulting in **a paper which appeared on arXiv this week**. One particular technique did us great service: *instance noise*. It's not the main focus of Casper's paper, so the details have been relegated to an Appendix. We thought it's a good idea to summarise it here, giving a few more details. Naturally, I think the full paper is also worth a read, there are a few more interesting things in there:

- Casper Kaae Sønderby, Jose Caballero, Lucas Theis, Wenzhe Shi and Ferenc Huszár (2016) **Amortised MAP Inference for Image Super-Resolution**

Instance noise

Summary

- we think a major reason for GANs' instability may be that the generative distributions are weird, degenerate, and

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

their support don't generally overlap with the true data distribution.

- this makes the nice theory break down and may lead to unstable behaviour
- we suggest adding noise to both real and synthetic data during training might help overcome these problems
- in this note we motivate this technique and illustrate in a few figures how it helps training

GANs should work.

There are different ways to think about GANs: you can approach it from a game theoretic view of seeking Nash equilibrium (**Salimans et al, 2016**), or you can treat it as an E-M like iterative algorithm where the discriminator's job is likelihood ratio estimation (**Mohamed et al, 2016, Uehara et al, 2016, Nowozin et al**). If you've read **my earlier posts**, it should come as no surprise that I subscribe to the latter view.

Consider the following idealised GAN algorithm, each iteration consisting of the following steps:

- we train the discriminator D via logistic regression between our generative model q_θ vs true data p , until convergence
- we extract from D an estimate of the logarithmic likelihood ratio
$$s(y) = \log \frac{q_\theta(y)}{p(y)}$$
- we update θ by taking a stochastic gradient step with objective function $\mathbb{E}_{y \sim q_\theta} s(y)$

If q_θ and p are well-conditioned distributions in a low-dimensional space, this algorithm performs gradient descent on an approximation to the KL divergence, so it should converge.

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

So why don't they?

Crucially, the convergence of this algorithm relies on a few assumptions never really made explicit that don't always hold:

1. that the log-likelihood-ratio $\log \frac{q_{\theta}(y)}{p(y)}$ is finite, or
2. that the Jensen-Shannon divergence $\mathbf{JS}[q_{\theta}|p]$ is a well-behaved function of θ and
3. that the Bayes-optimal solution to the logistic regression problem is unique: there is a single optimal discriminator that does a much better job than any other classifier.

In the paper we argued that in real-world situations neither of these holds, mainly because q_{θ} and p are concentrated distributions whose support may not overlap. In image modelling, distribution of natural images p is often assumed to be concentrated on or around a lower-dimensional manifold. Similarly, q_{θ} is often degenerate by construction. The odds that the two distributions share support in high-dimensional space, especially early in training, are very small.

If q_{θ} and p have non-overlapping support, then

1. the log-likelihood-ratio and therefore KL divergence is infinite and not well defined
2. the Jensen-Shannon divergence is saturated so its maximum value: To see why, consider the **mutual information interpretation of JS divergence**. If the two distributions q_{θ} and p have no overlap, they can be separated perfectly, so mutual information is maximal. If this is the case, $\mathbf{JS}$ is locally constant in θ .

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

3. the discriminator is extremely prone to overfitting: there may be a large set of near-optimal discriminators whose loss is very close to the Bayes optimum. Thus, for a fixed q_θ and p , training the discriminator D might lead to a different near-optimal solution each time depending on initialisation. And, each of these near-optimal solutions might provide very different gradients (or no useful gradients at all) to the generator.

How to fix this?

The main ways to avoid these pathologies involve making the discriminator's job harder. Why? The JS divergence is constant locally in θ , but it doesn't mean that **the variational lower bound** also has to be constant. Indeed, if you *cripple* the discriminator so the lower bound is not tight, you may end up with a non-constant function of θ that will roughly guide you to the right direction.

An example of this crippling is that in most GAN implementations the discriminator is only partially updated in each iteration, rather than trained until convergence. This extreme form of early stopping is a form of regularisation that prevents the discriminator from overfitting.

Another way to cripple the discriminator is adding label noise, or equivalently, *one-sided label smoothing* as introduced by **Salimans et al, (2016)**. In this technique the labels in the discriminator's training data are randomly flipped. Let's illustrate this technique in two figures.

The classifiaction view:

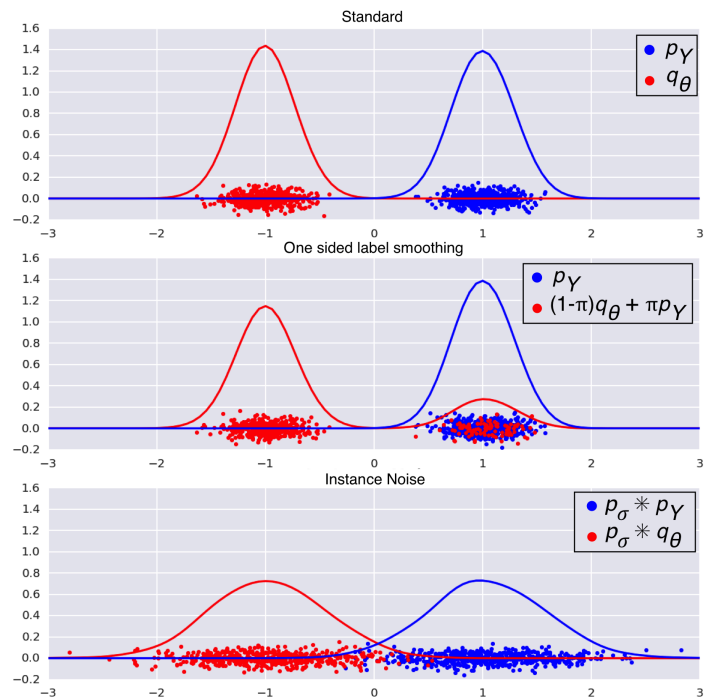
inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog



The top panel shows two almost perfectly separable distributions q_θ and p (it is called p_Y in the paper). Notice how the large gap between the distributions means that there are large number of possible classifiers that tell the two distributions apart and achieve similar logistic loss. The Bayes-optimal classifier may not be unique, and the set of near-optimal classifiers is very large and diverse.

In the middle panel we show the effect of `\emph{one sided label smoothing}` or equivalently, adding label noise. In this technique, the labels of some real data samples $x \sim p$ are flipped so the discriminator is trained thinking they were samples from q_θ . The discriminator indeed has a harder task now. However, the likelihood ratio $\log \frac{p}{(1-\pi)q_\theta + \pi p}$ is still not well-defined. Also, although discriminators have a harder job, they are all punished evenly: there is no way for the discriminator to be smart about handling label noise. Adding label noise doesn't change the structure of the logistic regression loss landscape dramatically, it mainly just pushes everything up. Hence, there are still a large

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

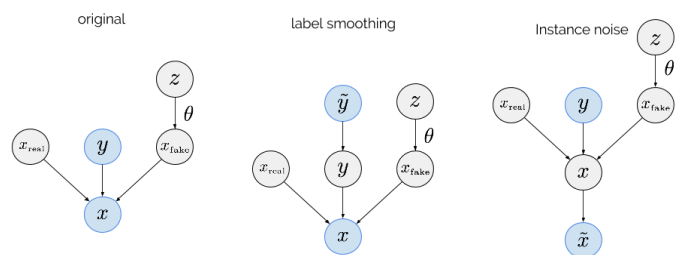
Blog

number of near-optimal discriminators. Adding label noise still does not allow us to pinpoint a single unique Bayes-optimal classifier. The JS divergence is not saturated to its maximum level anymore, but it is still locally constant in θ .

The last panel shows the technique we propose, whereby we add noise to samples from both q_θ and p . We use convolution $p_\sigma * q_\theta$ to denote additive noise. As a result, the noisy distributions now overlap, the log-likelihood-ratio $\log \frac{p_\sigma * p}{p_\sigma * q_\theta}$ is well-behaved, and the JS divergence between the two noisy distributions is a non-constant function of θ .

The graphical model view:

An alternative way to think about instance noise vs. label noise is via graphical models. The following three graphical models define joint distributions, parametrised by θ . The GAN algorithm tries to adjust θ so as to minimise the mutual information between the highlighted nodes in these graphical models:



Here's what the variables are:

- z is the latent variable we feed into the generator (in our image superresolution, this was the low-resolution image)
- x_{fake} is a synthetic sample we draw from the generator q_θ , by squashing z through a nonlinearity parametrised by θ
- x_{real} is a real data sample, drawn from p

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

- y is a binary label, a coinflip, sampled from a Bernoulli distribution with parameter 0.5
- x is a datapoint that the discriminator would see: depending on y , it's either a copy of x_{real} or x_{fake}
- $\tilde{y}$ is a noisy label that is the same as y most of the time, but they can be randomly flipped.
- $\tilde{x}$ is a noisy version of x , generated by adding noise $\nu \sim p_\sigma$ to x .

In this joint distribution, vanilla GANs minimise is the mutual information $\mathbb{I}[x, y]$ which corresponds to JS divergence. If the distributions of x_{fake} and x_{real} have no overlap, y is a deterministic function of x and therefore the mutual information is maximal. Hence, in this scenario, the objective function is theoretically constant in θ .

The second panel shows the effect of label smoothing, or, adding label noise. Now the discriminator is trained on randomly flipped labels $\tilde{y}$ instead of the real labels y . The GAN algorithm can be thought of as trying to minimise $\mathbb{I}[x, \tilde{y}]$. It is not hard to see that, in situations where y is a deterministic function of x , then this mutual information is also constant with respect to θ .

In the instance noise trick, the discriminator sees the correct labels y , but its input is the noisy $\tilde{x}$. We think that $\mathbb{I}[\tilde{x}, y]$ is a much better objective function to target. Even when $\mathbb{I}[y; x]$ is saturated, $\mathbb{I}[\tilde{x}, y]$ still depends on θ in a non-trivial and meaningful way. If the noise distribution p_σ is something like a Gaussian, adding noise can be thought of as a way to measure how far away q and p are from each when they don't overlap.

The divergence view

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

Finally, there's a way to understand label noise from the perspective of minimising divergences. Using a GAN algorithm we can construct algorithms that minimise the following family of divergences:

$$d_{\sigma}(q_{\theta}|p) = KL[p_{\sigma} * q_{\theta}|p_{\sigma} * p],$$

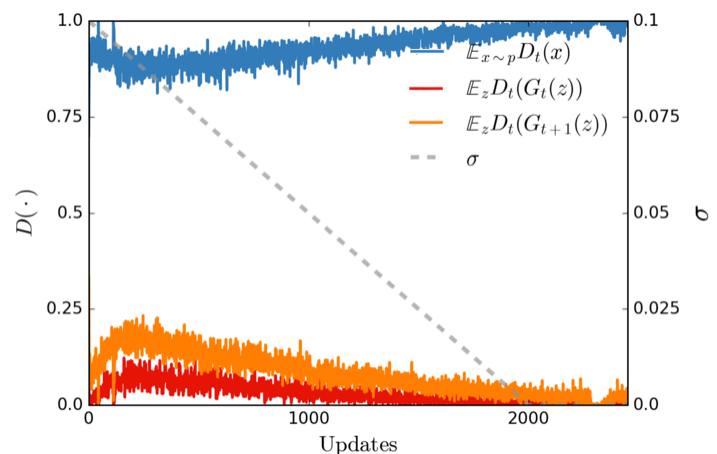
or

$$d_{\sigma,JS}(q_{\theta}|p) = JS[p_{\sigma} * q_{\theta}|p_{\sigma} * p],$$

where σ is the parameter of the noise distribution. Importantly, these divergences are still convex and have a global minimum when $q_{\theta} = p$. Obviously, as the noise level σ increases, the fine details of the distribution are "blurred out" so the divergences are expected to become less sensitive to fine patterns.

Does it work?

We tried this technique in the context of GANs for image superresolution, and we found it stabilises training, as predicted. We used additive Gaussian white noise whose variance parameter σ we annealed linearly during training. The figure below shows that the discriminator's performance is kept in check by the added noise throughout training:



inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space

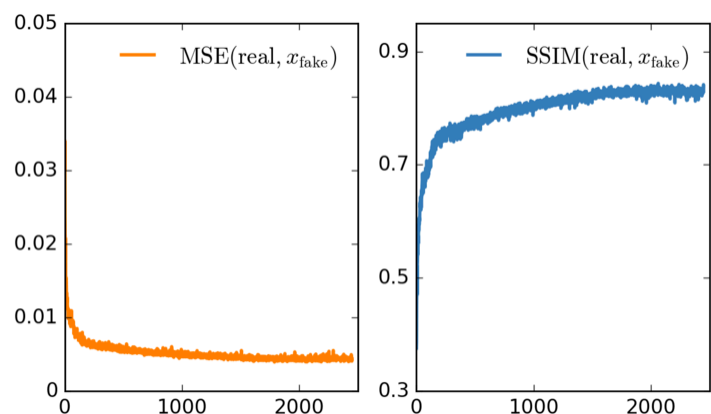


My lab

Blog

The blue curve shows the average probability the discriminator assigns to real data, the red the probability it assigns to synthetic data. If the discriminator was winning, these probabilities would always be **0.0** and **1.0**, but the noise makes the discriminator's job harder. The orange curve shows the probability the current discriminator D_t assigns to new fake data, after the generator is updated (G_{t+1}). As expected, the orange curve is always above the red one, which means that the current approximation of $d_\sigma(q_\theta|p)$ had been improved.

Another way to show that convergence is happening is to look at the average SSIM and MSE values during training. This is only possible because we trained the network for superresolution, so we actually always had a ground truth image to compare to. Both metrics improve steadily as the figures show:



Conclusion

Instance noise is a theoretically motivated way to remedy the poor convergence properties of GANs. We have not tested it extensively in the context of generative modeling, but we think it should help there also. Let us know if you have any experience with similar techniques.

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

ALSO ON INFERENCE

7 years ago · 7 comments

**The secular
Bayesian: Using
belief ...**

8 years ago · 1 comment

**Online Bayes
Deep Learning
...**

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



[My lab](#)

[Blog](#)

15 Comments

1 Login ▼

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

♡ 12 Share

Best Newest Oldest

E

Etienne Vos

8 years ago

@Ferenc Huszar Thanks for the great post. One very simple (maybe unrelated) question that I have is: Should the output of $D(\cdot)$ for real and fake samples converge to 1 and 0, respectively, as in your figure showing the annealing of sigma? Or should $D(\cdot)$ converge to 0.5 for both real and fake... can't seem to grasp what is the preferred behaviour.

1 0 Reply ↗



Ferenc Huszar Mod

→ Etienne Vos

8 years ago

So, ideally, your generator should match the data distribution exactly, at which point the discriminator D should be unable to differentiate between the two classes, so it should output 0.5. This is the desired long-term outcome.

However, the generator can never be precisely the same as the true data distribution (model misspecification) so this state will probably never be reached. What's worse, because both the data

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

distribution and the generator distribution tend to be highly concentrated on some manifolds, their actual KL divergence will be infinite, and in this case the discriminator would most likely converge to the 0-1 behaviour.

Adding instance noise prevents this 0-1 trivial solution from happening, because the noise-infused distributions always have finite KL divergence. Over time, if your generator can describe the data well, you should see the discriminator to converge or at least hover around very poor performance, i.e. outputting 0.5 most of the time.

Note that one interpretation of the Wasserstein GAN is that it avoids the 0-1 issue by forcing a Lipschitz-constraint on the discriminator, thereby making sure that the regularized discriminator is always informative.

5 0 Reply



Ruth Smalbraak



4 years ago

Hi, thank you for this post! It has greatly helped me in understanding these GAN "tricks".

I know this post was posted quite a while ago, but I was wondering the following; in your post you state that "In one-sided label smoothing ... the labels of some real data samples $x \sim p$ are flipped so the discriminator is trained thinking they were samples from q_θ ", meaning the labels of some real instances would be flipped from $y=1$ to $y=0$. However, what I understood from the definition of "label smoothing" in Salimans et al., (2016), is that positive labels ($y=1$) can be replaced with a value like ($y=0.9$), thus "softening" the positive labels. Aren't these two different definitions of label smoothing? And if so, can they both be

inFERENCe

posts on machine learning,
statistics, opinions on things
I'm reading in the space



My lab

Blog

applied?

Thank you in advance!

0 0 Reply

S

Shengzhao Lei

6 years ago

Does it also work well if I use wGAN-gp with instance noise?

0 0 Reply

T

Tony

8 years ago

Very interesting post! I would like to confirm whether in your super-resolution example, you directly add Gaussian noise to the real and fake images and then feed them into the discriminator? How about jittering the image a little bit? How is the training without the instance noise technique? Thanks!

0 0 Reply

Z

zheng

9 years ago edited

What is the x-axis in the second last figure? the training progress for the discriminator with generator fixed? Is it different from the x-axis in the last figure, where I guess is the training progress for the entire network (i.e. both generator and discriminator)?

0 0 Reply



bobchennan

10 years ago

How to check whether GAN converges or not?

In the idea case according to Nash

inFERENCe

posts on machine learning,
statistics, opinions on things

I'm reading in the space



My lab

Blog